

# Curve curvature in numpy

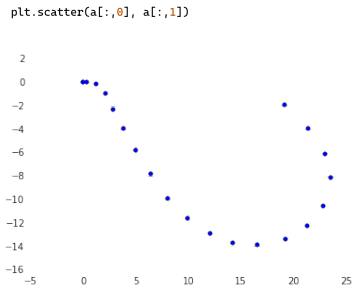
Asked 9 years, 10 months ago Modified 1 year, 9 months ago Viewed 39k times

I am measuring x,y coordinates (in cm) of an object with a special camera in fixed time intervals of 1s. I have the data in a numpy array:

```
26 a = np.array([[ 0. ,  0. ],[ 0.3 ,  0. ],[ 1.25, -0.1 ],[ 2.1 , -0.9 ],[ 2.85, -2.3 ],[ 3.8 , -3.95],[ 5. , -5.75],[ 6.4 , -7.8 ],[ 8.05, -9.9 ],[ 9.9 , -11.6 ],[ 12.05, -12.85],[ 14.25, -13.7 ],[ 16.5 , -13.8 ],[ 19.25, -13.35],[ 21.3 , -12.2 ],[ 22.8 , -10.5 ],[ 23.55, -8.15],[ 22.95, -6.1 ],[ 21.35, -3.95],[ 19.1 , -1.9 ]])
```



And the curve looks like this:



## Question:

How can I calculate the tangential and the radial aceleration vectors at each point? I found some formulas that might be relevant:

- Speed =  $\frac{ds}{dt} = |\mathbf{v}(t)| = \sqrt{(x')^2 + (y')^2}$ .
- $\mathbf{v} = \frac{ds}{dt} \mathbf{T}$ ,  $\mathbf{T} = \frac{\mathbf{v}}{ds/dt}$
- $\mathbf{a}(t) = \frac{d^2s}{dt^2} \mathbf{T} + \kappa \left( \frac{ds}{dt} \right)^2 \mathbf{N} = \frac{d^2s}{dt^2} \mathbf{T} + \frac{v^2}{R} \mathbf{N}$
- $\kappa = \frac{d\mathbf{T}}{ds} = \frac{|\mathbf{a} \times \mathbf{v}|}{|\mathbf{v}|^3}$ .
- 4a. For plane curves  $\mathbf{r}(t) = x(t)\hat{\mathbf{i}} + y(t)\hat{\mathbf{j}}$ :  $\kappa = \frac{|x''y' - x'y''|}{((x')^2 + (y')^2)^{3/2}}$ .

I am able to easily calculate the  $v_x$  and the  $v_y$  projections with `np.diff(a, axis=0)` but I am a numpy/python noob and it is way over my head to continue. If I could calculate the curvature at each point, also my problem would be solved. Can somebody help?

python numpy Edit tags

Share Edit Follow Close Flag Protect

edited Feb 2, 2015 at 1:16

asked Feb 1, 2015 at 23:55

Elian  
447 1 5 11

1 Well, unless you have a nice system of parametric equations for the curve (which I don't believe you do), you'll have to replace, eg  $x'$  with  $(\Delta x)/(\Delta t)$  (forgive the crude math notation, since SO doesn't support LaTeX). Since your intervals are all one second apart,  $\Delta t$  is 1, so you can replace  $x'$  with  $\Delta x$  and likewise with  $y'$ . These will be approximations, of course, but unless you attempt some curve-fitting, that's the best you can do with the dataset you have.  
– user554546 Feb 2, 2015 at 0:12

Oops, sorry, my answer is a bit too long-winded, now that you're only after curvature. Hopefully, it's still helpful. – user554546 Feb 2, 2015 at 2:34

## 4 Answers

Sorted by: Date modified (newest first)

[Reset to default](#)



The [answer](#) looks good for the most part but I would correct the following:

0

Change



```
tangent = np.array([1/ds_dt] * 2).transpose() * velocity
```

to



```
tangent = np.array([1/ds_dt]).transpose() * velocity
```

And

```
normal = np.array([1/length_dT_dt] * 2).transpose() * dT_dt
```

to

```
normal = np.array([1/length_dT_dt]).transpose() * dT_dt
```

The Tangent is the velocity normalized, there is no need to multiply by 2.

Share Edit Follow Flag

answered Mar 9, 2023 at 16:07

Pat  
668 1 9 25



**EDIT:** I put together this answer off and on over a couple of hours, so I missed your latest edits indicating that you only needed curvature. Hopefully, this answer will be helpful regardless.

60

Other than doing some curve-fitting, our method of approximating derivatives is via [finite differences](#). Thankfully, `numpy` has a [gradient](#) method that does these difference calculations for us, taking care of the details of averaging previous and next slopes for each interior point and leaving each endpoint alone, etc.



```
import numpy as np
```



```
a = np.array([[ 0. ,  0. ],[ 0.3 ,  0. ],[ 1.25, -0.1 ],[ 2.1 , -0.9 ],[ 2.85, -2.3 ],[ 3.8 , -3.95],[ 5. , -5.75],[ 6.4 , -7.8 ],[ 8.05, -9.9 ],[ 9.9 , -11.6 ],[ 12.05, -12.85],[ 14.25, -13.7 ],[ 16.5 , -13.8 ],[ 19.25, -13.35],[ 21.3 , -12.2 ],[ 22.8 , -10.5 ],[ 23.55, -8.15],[ 22.95, -6.1 ],[ 21.35, -3.95],[ 19.1 , -1.9 ]])
```

Now, we compute the derivatives of each variable and put them together (for some reason, if we just call `np.gradient(a)`, we get a list of arrays...not sure off the top of my head what's going on there, but I'll just work around it for now):

This gives us the following vector for `velocity`:

which makes sense when glancing at the scatterplot of  $a$ .

```
ds_dt = np.sqrt(dx_dt * dx_dt + dy_dt * dy_dt)
```

```
array([ 0.3, 0.62699681, 1.00623059, 1.36014705, 1.74588803,
       2.03254766, 2.32284847, 2.57512136, 2.58311827, 2.48508048,
       2.41518633, 2.27513736, 2.50611752, 2.52982213, 2.27623593,
       2.31651678, 2.20127804, 2.37065392, 2.8487936, 3.04384625])
```

```
tangent = np.array([1/ds_dt] * 2).transpose() * velocity
```

```
array([[ 1.,          0.,
        0.99681528, -0.07974522,
        0.89442719, -0.44712136,
        0.5881717,   -0.38873608,
        0.48685826, -0.37348099,
        0.5289289,  -0.84668859,
        0.55965769, -0.82872388,
        0.9252051,  -0.88578727,
        0.67747575, -0.73554511,
        0.84088291, -0.59354215,
        0.90655164, -0.34749077,
        0.97796253, -0.2687786,
        0.99755897,  0.66982913,
        0.9486833,   0.31622777,
        0.77797614,  0.62603352,
        0.85642639,  0.87415728,
        0.83487112,  0.99941941,
        0.64006099,  0.88583154,
        0.67572463,  0.73715414,
        0.73919634,  0.67349   ]])
```

In [12]:

```
In [12]: np.sqrt(tangent[:,0] * tangent[:,0] + tangent[:,1] * tangent[:,1])
```

```
Out[12]:
```

```
array([ 1.,  1.,  1.,  1.,  1.,  1.,  1.,  1.,  1.,  1.,  1.,  1.,  1.,  
        1.,  1.,  1.,  1.,  1.,  1.,  1.] )
```

```
tangent_x = tangent[:, 0]
tangent_y = tangent[:, 1]

deriv_tangent_x = np.gradient(tangent_x)
deriv_tangent_y = np.gradient(tangent_y)

dT_dt = np.array([ deriv_tangent_x[i], deriv_tangent_y[i]] for i in
orange(deriv_tangent_x.size)])

length_dT_dt = np.sqrt(deriv_tangent_x * deriv_tangent_x + deriv_tangent_y *
deriv_tangent_y)

normal = np.array([1/length_dT_dt * 2]).transpose() * dT_dt
```

```
array([[ -0.83990439, -0.99920303],
       [-0.22975292, -0.97234899],
       [-0.48897529, -0.87227953],
       [-0.69187665, -0.72781677],
       [-0.8292422, -0.55888941],
       [-0.5188065, -0.52373629],
       [-0.8278434, -0.56695927],
       [-0.78434982, -0.62031876],
       [-0.70763955, -0.76561953],
       [-0.59568265, -0.80321988],
       [-0.41839796, -0.9106993],
       [-0.18879684, -0.98201617],
       [-0.5568352, -0.99840847],
       [-0.36457812, -0.93117594],
       [-0.63863584, -0.76509111],
       [-0.89417603, -0.444771557],
       [-0.99992445, -0.8122923],
       [-0.93801622, -0.34659137],
       [-0.79170994, -0.61808935],
       [-0.70603568, -0.70817626])
```

Note that the normal vector represents the direction in which the curve is turning. The vector above then makes sense when viewed in conjunction with the scatterplot for  $\mathbf{a}$ . In particular, we go from turning down to turning up after the fifth point, and we start turning to the left (with respect to the  $x$  axis) after the 12th point.

Finally, to get the tangential and normal components of acceleration, we need the second derivatives of  $s$ ,  $x$ , and  $y$  with respect to  $t$ , and then we can get the curvature and the rest of our components (keeping in mind that they are all scalar functions of  $t$ ):

```
d2s_dt2 = np.gradient(ds_dt)
d2x_dt2 = np.gradient(dx_dt)
d2y_dt2 = np.gradient(dy_dt)

curvature = np.abs(d2x_dt2 * dy_dt - dx_dt * d2y_dt2) / (dx_dt * dx_dt + dy_dt * dy_dt)**1.5
t_component = np.array([d2s_dt2] * 2).transpose()
n_component = np.array([curvature * ds_dt * ds_dt] * 2).transpose()

acceleration = t_component * tangent + n_component * normal
```

Share Edit Follow Flag

edited Feb 2, 2015 at 2:39

answered Feb 2, 2015 at 2:34  
user554546

2 ▲ Thanks a lot for helping out the community @JackManey! – Olshansky Mar 11, 2016 at 5:59

▲ Hi @JackManey,Nice answer. I have one doubt that what does the negative values indicate for normal vector.Shall i add them in 'a' for getting the actual values of normal to curve at point? – Gunjan naik Sep 9, 2016 at 20:42

▲ @user3515225: Thanks. As to your question...well, the normal vector is a vector. It's allowed to have negative coordinates. This isn't an issue. – user554546 Sep 9, 2016 at 21:00

▲ Isn't this calculation an overkill? As far as I understand, in order to obtain the curvature it suffices to compute the expression  $\text{length}_{dT} / ds_{dt}$ . – kiriloso Aug 27, 2017 at 12:56

▲ This is an amazing post! Thank you very much! The only part that got me confused is the "t\_component" and the "n\_component" bits. When I plot the curvature I get the right thing (I guess). What are the "t\_component", "n\_component" and the "acceleration" bits for? Thanks. – mbilyanov Dec 27, 2017 at 15:08



-1



🔒 This post is hidden. It was deleted 6 years ago by Jean-François Fabre ♦, Jon Clements.

@user554546: Thanks for your insightful response to computing curvature for 2D points. Please I am curious to find out if it is possible to generalize your technique for points in 3D, i.e point cloud with  $[x, y, z]$  coordinates? Your response or examples would be of immense benefit. Regards

Share Edit Follow Undelete Flag

answered Apr 9, 2018 at 17:39

Zones Two  
1

▲ This type of answer is more suited towards a comment as it does not directly solve the question – liam Apr 9, 2018 at 17:43

▲ This does not provide an answer to the question. Once you have sufficient reputation you will be able to comment on any post; instead, provide answers that don't require clarification from the asker. – Zoe - Save the data dump ♦ Apr 9, 2018 at 18:06

Comments disabled on deleted / locked posts / reviews



0



🔒 This post is hidden. It was deleted 6 years ago by josliber.

Comment on post by user554546: Great answer. I have a 3-dimensional situation and would like to compute curvature. How do you modify the computation of curvature to include  $dz_{dt}$  and  $d2z_{dt2}$ ? And is there a reference to this computation of curvature?

Share Edit Follow Undelete Flag

edited Apr 5, 2018 at 14:09

answered Apr 5, 2018 at 14:00

mkoen  
1 1

1 ▲ This does not provide an answer to the question. You can search for similar questions, or refer to the related and linked questions on the right-hand side of the page to find an answer. If you have a related but different question, ask a new question, and include a link to this one to help provide context. See: Ask questions, get answers, no distractions – Natty Apr 5, 2018 at 14:01

▲ If you have a new question, please ask it by clicking the Ask Question button. Include a link to this question if it helps provide context. - From Review – André Kool Apr 5, 2018 at 14:22

▲ This does not provide an answer to the question. Once you have sufficient reputation you will be able to comment on any post; instead, provide answers that don't require clarification from the asker. - From Review – EJoshuaS - Stand with Ukraine Apr 5, 2018 at 15:05

▲ Please take the site tour - Stack Overflow is not a discussion forum, so you're not permitted to use the Answer section to comment on other posts or ask follow-up questions. – EJoshuaS - Stand with Ukraine Apr 5, 2018 at 15:07

Comments disabled on deleted / locked posts / reviews